❶ System that provides the materialization resources
❷ Source
❸ Flow
❹ Sink
❺ Blueprint
❻ Execution
❼ Terminates the ActorSystem

10.1.8 Infinite streams

Examples of infinite streams are tweets received through an endpoint, updates read from a database from a table representing new orders on a website, and events from a car's telemetry read from a Kafka topic. Let's look at a scenario similar to the previous one. The only difference is that the data source is infinite, randomly produces integers from 0 to 3 (not including 3), and has a materialized value `Cancellable` that can be used to cancel the stream. This cancellable value traverses the stream until the `Sink`.

The next listing shows all of this in compact stream form in the next listing. Note that each text describing the test is unique; otherwise, the spec wouldn't compile. This is an infinite producer with a consumer that creates a side effect.

Listing 10.2 Cancelling an infinite stream

```
import akka.stream.scaladsl.{
  Keep,
  Sink,
  Source
}

import akka.Done
import akka.actor.Cancellable

import scala.concurrent.{ Await, Future }
import scala.concurrent.duration._

class StreamsSpec ... {

  ...

  "an infinite producer with a consumer creating side effect" should {
    "be cancellable" in {

      var fakeDB: List[Int] = List()                          ❶
      def storeDB(value: Int) =
          fakeDB = fakeDB :+ value                            ❶

      val liveSource: Source[Int, Cancellable] =
        Source.tick(1.second, 1.second, Random.nextInt(3))    ❷

      val (cancellable, future): (Cancellable, Future[Done]) =
        liveSource.filter(_ % 2 == 0)                         ❸
          .toMat(Sink.foreach(storeDB))(Keep.both)            ❹
          .run

      Thread.sleep(3000)                                      ❺
      cancellable.cancel

      Await.result(future, 1.seconds)                         ❻
      assert(future.isCompleted)
    }
```